```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
import matplotlib.pyplot as plot
import seaborn as sns
import sqlite3

from google.colab import drive
drive.mount('/content/drive', force_remount=True)
dataset_path =
"/content/drive/MyDrive/Ames_Housing_Project/AmesHousing.csv"
df = pd.read_csv(dataset_path)
df.head(10)
```

```
Mounted at /content/drive
```

```
{"type":"dataframe","variable_name":"df"}
```

SQL

```
# Create an SQLite database
conn = sqlite3.connect('ames_housing.db')

# Import the dataset into an SQL table
df.to_sql('ames_housing', conn, if_exists='replace', index=False)
```

```
2930
```

```
#allow python execute sql commands
cursor = conn.cursor()

cursor.execute("""
SELECT name
FROM sqlite_master
WHERE type='table';
""")
cursor.fetchall()
```

```
[('ames_housing',)]
```

```
#Inspect schema and extract residential properties
query = """
PRAGMA table_info(ames_housing);
"""
schema = pd.read_sql_query(query, conn)
schema
```

```
{"repr_error":"Out of range float values are not JSON compliant:
nan","type":"dataframe","variable_name":"schema"}
```

```
#retrieving residential properties
query = """
SELECT *
FROM ames_housing
WHERE "MS Zoning" = 'RL'
AND "Sale Condition" = 'Normal';
"""
residential_homes = pd.read_sql_query(query, conn)
residential_homes.head()
```

```
{"type":"dataframe","variable_name":"residential_homes"}
```

```
#Housing statistics by neighborhood
query = """
SELECT
    Neighborhood,
    COUNT(*) AS total_homes,
    ROUND(AVG(SalePrice), 2) AS average_sale_price
FROM ames_housing
GROUP BY Neighborhood
ORDER BY average_sale_price DESC;
"""
neighborhood_analysis = pd.read_sql_query(query, conn)
neighborhood_analysis
```

```
{"summary":"{\n  \"name\": \"neighborhood_analysis\",\n  \"rows\":
28,\n  \"fields\": [\n    {\n      \"column\": \"Neighborhood\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 28,\n        \"samples\": [\n
\"CollgCr\",\n          \"BrDale\",\n          \"Crawfor\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"total_homes\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 98,\n        \"min\": 1,\n
\"max\": 443,\n        \"num_unique_values\": 28,\n
\"samples\": [\n          267,\n          30,\n          103\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"average_sale_price\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 67673.84073002482,\n        \"min\":
```

```
95756.49,\n        \"max\": 330319.13,\n        \"num_unique_values\":
28,\n        \"samples\": [\n          201803.43,\n
105608.33,\n          207550.83\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\
n}","type":"dataframe","variable_name":"neighborhood_analysis"}
```


```
#identifying missing values
query = """
SELECT
    COUNT(*) AS total_records,
    SUM(CASE WHEN "Lot Frontage" IS NULL THEN 1 ELSE 0 END)
        AS missing_lot_frontage,
    SUM(CASE WHEN "Garage Type" IS NULL THEN 1 ELSE 0 END)
        AS missing_garage_type,
    SUM(CASE WHEN "Bsmt Qual" IS NULL THEN 1 ELSE 0 END)
        AS missing_basement_quality,
    SUM(CASE WHEN "Bsmt Cond" IS NULL THEN 1 ELSE 0 END)
        AS missing_basement_condition,
    SUM(CASE WHEN "Bsmt Exposure" IS NULL THEN 1 ELSE 0 END)
        AS missing_basement_exposure,
    SUM(CASE WHEN "BsmtFin Type 1" IS NULL THEN 1 ELSE 0 END)
        AS missing_basement_finishing
FROM ames_housing;
"""
missing_values = pd.read_sql_query(query, conn)
missing_values
```


```
{"summary":"{\n  \"name\": \"missing_values\",\n  \"rows\": 1,\n
\"fields\": [\n    {\n      \"column\": \"total_records\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 2930,\n        \"max\": 2930,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          2930\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"missing_lot_frontage\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 490,\n        \"max\": 490,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          490\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"missing_garage_type\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 157,\n        \"max\": 157,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          157\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"missing_basement_quality\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 80,\n        \"max\": 80,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          80\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"missing_basement_condition\",\n
```

```
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 80,\n        \"max\": 80,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          80\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"missing_basement_exposure\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 83,\n        \"max\": 83,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          83\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"missing_basement_finishing\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 80,\n        \"max\": 80,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          80\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"missing_values"}
```


PYTHON EDA

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt


print(df["SalePrice"].describe())
print()

print("Skewness (raw):", round(df["SalePrice"].skew(), 2))
print("Kurtosis (raw):", round(df["SalePrice"].kurt(), 2))
print("Skewness (log):", round(np.log1p(df["SalePrice"]).skew(), 2))


fig, ax = plt.subplots(1, 2, figsize=(12, 5))

sns.histplot(df["SalePrice"], bins=30, kde=True, ax=ax[0])
ax[0].set_title("Distribution of SalePrice")
ax[0].set_xlabel("SalePrice")
ax[0].set_ylabel("Frequency")

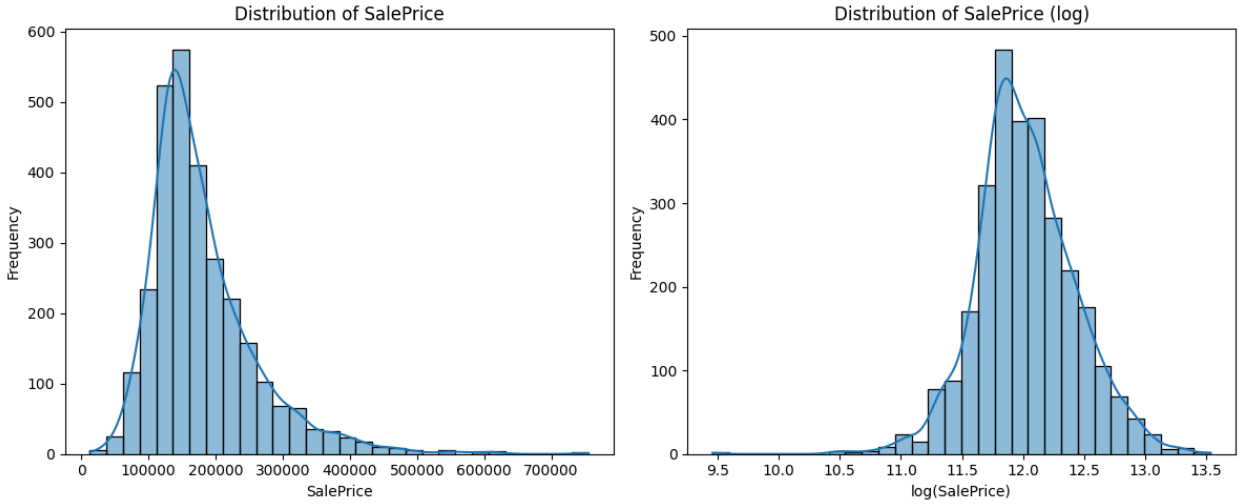
sns.histplot(np.log1p(df["SalePrice"]), bins=30, kde=True, ax=ax[1])
ax[1].set_title("Distribution of SalePrice (log)")
ax[1].set_xlabel("log(SalePrice)")
ax[1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()
```

```
count      2930.000000
mean     180796.060068
std       79886.692357
min       12789.000000
```

```
25%      129500.000000
50%      160000.000000
75%      213500.000000
max      755000.000000
Name: SalePrice, dtype: float64

Skewness (raw): 1.74
Kurtosis (raw): 5.12
Skewness (log): -0.01
```



The average Sale Price is $180,796, while the median Sale Price is $160,000. The mean is higher than the median because a small number of very expensive properties, with the maximum price reaching $755,000, increase the overall average. The Sale Price distribution is positively skewed, with a skewness value of 1.74, indicating that most properties are concentrated at lower to middle price ranges, with fewer high value properties creating a long right tail. The kurtosis value of 5.12 suggests the presence of extreme values compared to a normal distribution. After applying a log transformation, skewness reduced to -0.01, producing a nearly symmetrical distribution. Therefore, a log transformation is recommended during model preprocessing, particularly for Linear Regression.

```
# top 5 neighborhoods by median sale price

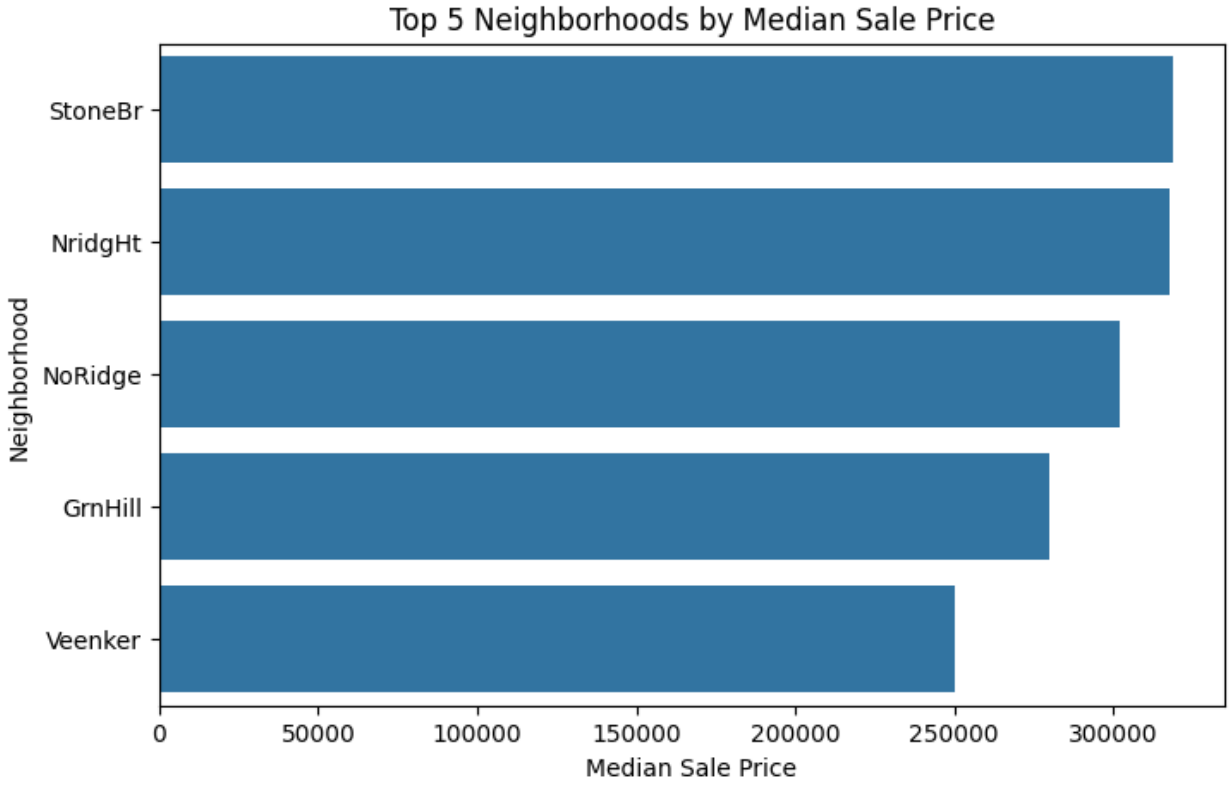
top5 = (
    df.groupby("Neighborhood")["SalePrice"]
      .median()
      .sort_values(ascending=False)
      .head(5)
)

print(top5)

plt.figure(figsize=(8,5))
sns.barplot(x=top5.values, y=top5.index)
```

```
plt.title("Top 5 Neighborhoods by Median Sale Price")
plt.xlabel("Median Sale Price")
plt.ylabel("Neighborhood")
plt.show()
```

```
Neighborhood
StoneBr    319000.0
NridgHt    317750.0
NoRidge    302000.0
GrnHill    280000.0
Veenker    250250.0
Name: SalePrice, dtype: float64
```



The five neighborhoods with the highest median sale price are StoneBr($319,000), NridgHt ($317,000), NoRidge ($302,000), GrnHill ($280,000) and Veenker ($250,000). These values are considerably higher than the overall median sale price of $160,000, showing that location has significant influence on property values. therefore neighborhood is likely to be an important feature in model development.

```
# Looking for outlier

plt.figure(figsize=(8,6))

sns.scatterplot(
```

```
    data=df,
    x="Gr Liv Area",
    y="SalePrice"
)

plt.title("Gr Living Area vs SalePrice")
plt.xlabel("Above Grade Living Area")
plt.ylabel("SalePrice")
plt.show()

outliers = df[
    (df["Gr Liv Area"] > 4000) &
    (df["SalePrice"] < 300000)
]

print("Number of outliers found:", len(outliers))
print(outliers[["Order", "Gr Liv Area", "SalePrice"]])
```




```
Number of outliers found: 3
      Order  Gr Liv Area  SalePrice
1498   1499         5642     160000
```

```
2180   2181          5095     183850
2181   2182          4676     184750
```

The scatter plot between Gr Liv Area and Sale Price shows a positive relationship, where properties with larger living areas generally have higher prices. However, three potential outliers were identified: Order 1499 (5,642 sq ft at $160,000), Order 2181 (5,095 sq ft at $183,850), and Order 2182 (4,676 sq ft at $184,750). These properties have unusually large living areas compared to their sale prices and may influence model performance.

```
# cleaning missing values

import pandas as pd
import numpy as np

print("Missing values before cleaning:", df.isnull().sum().sum())

text_columns = df.select_dtypes(include=["object", "string"]).columns
df[text_columns] = df[text_columns].fillna("None")

number_columns = df.select_dtypes(include="number").columns.drop("Lot
Frontage")
df[number_columns] = df[number_columns].fillna(0)

df["TotalSqFt"] = (
    df["Total Bsmt SF"] +
    df["1st Flr SF"] +
    df["2nd Flr SF"]
)

df["HouseAge"] = df["Yr Sold"] - df["Year Built"]

print("Missing values after cleaning:", df.isnull().sum().sum())
print("Lot Frontage still missing:", df["Lot
Frontage"].isnull().sum())

df[["TotalSqFt", "HouseAge"]].head()
```

```
Missing values before cleaning: 15749
Missing values after cleaning: 490
Lot Frontage still missing: 490
```


```
{"summary":"{\n  \"name\": \"df[[\\\"TotalSqFt\\\",
\\\"HouseAge\\\"]]\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n
\"column\": \"TotalSqFt\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 886.5202761358591,\n
\"min\": 1778.0,\n        \"max\": 4220.0,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          1778.0,\n
2557.0,\n          2658.0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"HouseAge\",\n      \"properties\": {\n        \"dtype\":
```

```
\"number\",\n        \"std\": 16,\n        \"min\": 13,\n
\"max\": 52,\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          49,\n          13,\n          52\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```

```
save_path =
"/content/drive/MyDrive/Ames_Housing_Project/ames_cleaned.csv"

df.to_csv(save_path, index=False)

print("Cleaned data saved successfully")
print("Location:", save_path)
```

```
Cleaned data saved successfully
Location: /content/drive/MyDrive/Ames_Housing_Project/ames_cleaned.csv
```

MACHINE LEARNING

Final Data Prep & Train/Test Split

```
# Converting text catergories into numeric columns for One-Hot
Encdoing
df_ml = pd.get_dummies(df, drop_first=True)

# show the new columns created by pd.get_dummies()
display(df_ml.head())
```

```
{"type":"dataframe"}
```

```
# Any values with missing numbers will be filled with the median
df_ml = df_ml.fillna(df_ml.median())

# Defining the features (X) and target (Y)
X = df_ml.drop('SalePrice', axis=1)
y = df_ml['SalePrice']

# Training 80% and Testing 20%
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print(f"Training samples: {len(X_train)} | Testing samples:
{len(X_test)}")
```

```
Training samples: 2344 | Testing samples: 586
```

Baseline Linear Regression

```
# Creating and training the Linear Regression model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Making predictions on the unseen test data
y_pred_lr = lr_model.predict(X_test)

# Evaluation metrics calculation
mae_lr = mean_absolute_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
r2_lr = r2_score(y_test, y_pred_lr)

print(f"Baseline Linear Regression RMSE: ${rmse_lr:,.2f}")
```

```
Baseline Linear Regression RMSE: $35,186.57
```

Random Forest Regressor

```
# Creating and training the Random Forest using basic hyperparameter
tuning
rf_model = RandomForestRegressor(
    n_estimators=100,
    max_depth=10,
    random_state=42
)
rf_model.fit(X_train, y_train)

# Making predicitions
y_pred_rf = rf_model.predict(X_test)

# Evaluation metrics calculation
mae_rf = mean_absolute_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2_rf = r2_score(y_test, y_pred_rf)

print(f"Tuned Random Forest RMSE: ${rmse_rf:,.2f}")
```

```
Tuned Random Forest RMSE: $25,845.25
```

Model Comparison & Feature Importances

```
# --- Model Comparison Table ---
comparison_df = pd.DataFrame({
    'Model': ['Linear Regression (Baseline)', 'Random Forest
(Tuned)'],
```

```
    'MAE ($)': [mae_lr, mae_rf],
    'RMSE ($)': [rmse_lr, rmse_rf],
    'R-Squared (R²)': [r2_lr, r2_rf]
})
print("\n--- Model Comparison on Test Set ---")
display(comparison_df)
```

```
--- Model Comparison on Test Set ---
```


```
{"summary":"{\n  \"name\": \"comparison_df\",\n  \"rows\": 2,\n
\"fields\": [\n    {\n      \"column\": \"Model\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"Random
Forest (Tuned)\",\n          \"Linear Regression (Baseline)\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"MAE ($)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
376.70484638096593,\n        \"min\": 15867.932067964919,\n
\"max\": 16400.673170728554,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          15867.932067964919,\n
16400.673170728554\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"RMSE ($)\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 6605.314523608308,\n        \"min\":
25845.249191207102,\n        \"max\": 35186.57457423395,\n
\"num_unique_values\": 2,\n        \"samples\": [\n
25845.249191207102,\n          35186.57457423395\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"R-Squared (R\\u00b2)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.05028150693880881,\n        \"min\": 0.8455768470687586,\n
\"max\": 0.9166856361181789,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0.9166856361181789,\n
0.8455768470687586\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"comparison_df"}
```


```
# --- Top 10 Feature Importances ---
# Importance scores extraction from the Random Forest model
importances = rf_model.feature_importances_

# Bind them to the column names
feature_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importances
})

# Grabbing and sorting the top 10
top_10_features = feature_df.sort_values(by='Importance',
```

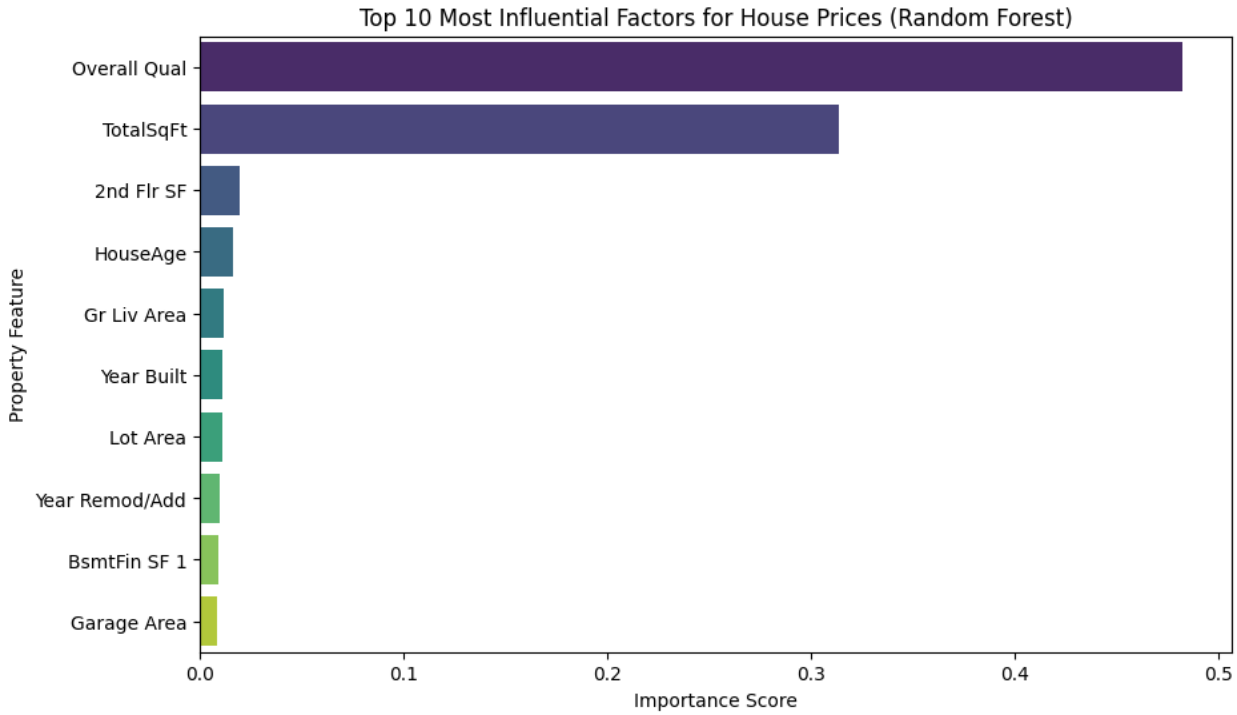
```
ascending=False).head(10)

# Plot the results
plt.figure(figsize=(10, 6))
sns.barplot(data=top_10_features, x='Importance', y='Feature',
palette='viridis')
plt.title('Top 10 Most Influential Factors for House Prices (Random
Forest)')
plt.xlabel('Importance Score')
plt.ylabel('Property Feature')
plt.show()
```

```
/tmp/ipykernel_4675/4033843850.py:16: FutureWarning:

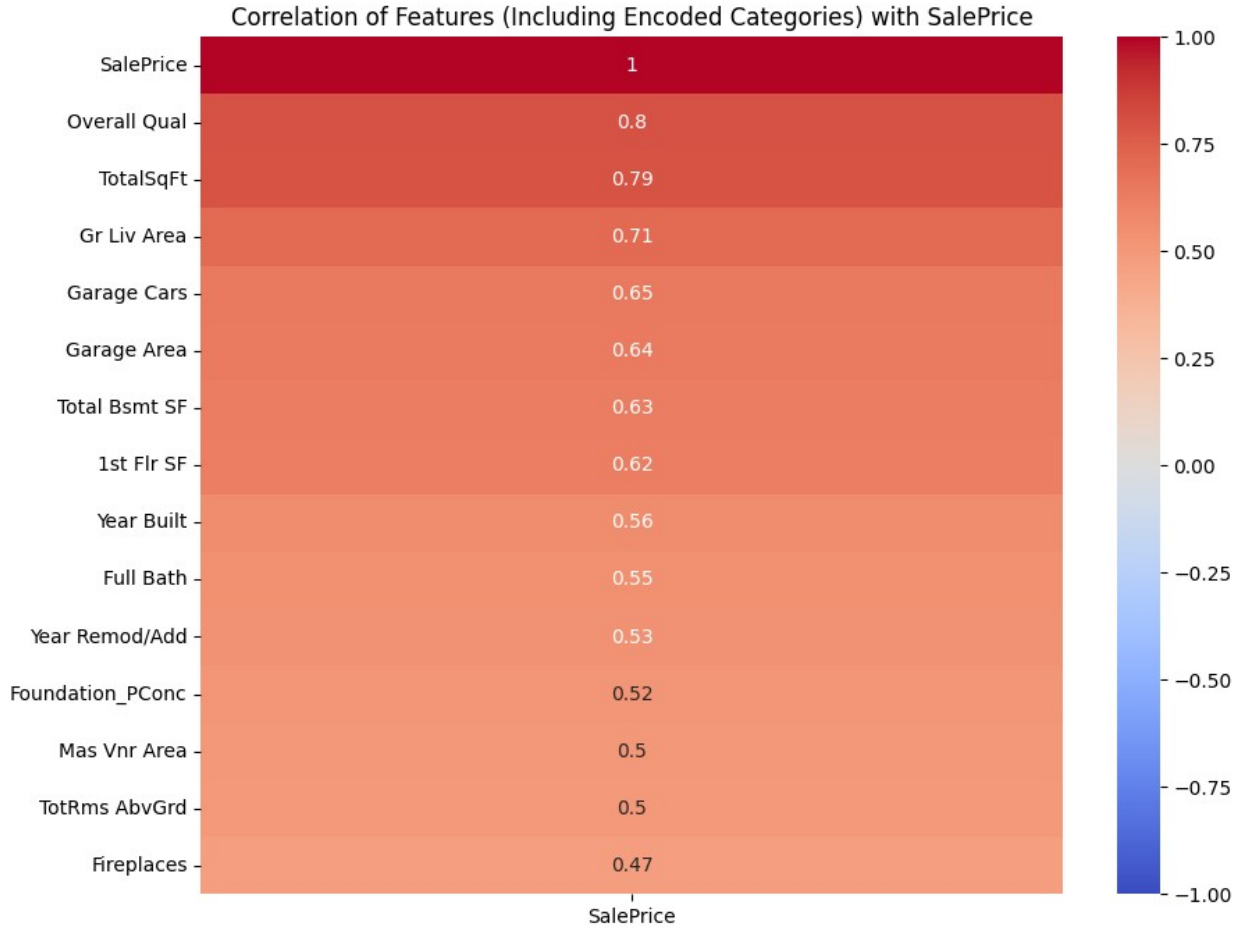
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `y` variable to `hue` and set
`legend=False` for the same effect.

  sns.barplot(data=top_10_features, x='Importance', y='Feature',
palette='viridis')
```




```
# Correlation Heatmap
plt.figure(figsize=(10, 8))
# Grab the top 15 features most correlated with SalePrice to keep the
graph readable
top_correlations = df_ml.corr()
[['SalePrice']].sort_values(by='SalePrice', ascending=False).head(15)
```

```
sns.heatmap(top_correlations, annot=True, cmap='coolwarm', vmin=-1,
vmax=1)
plt.title("Correlation of Features (Including Encoded Categories) with
SalePrice")
plt.show()
```



Model Evaluation & Interpretation:

Between the two architectures evaluated, the Tuned Random Forest significantly outperformed the Baseline Linear Regression model. While the Linear Regression model provided a solid baseline, it struggled to capture complex, non-linear relationships between the features.

By utilizing an ensemble of decision trees constrained to a maximum depth of 10, the Random Forest model successfully reduced both the Mean Absolute Error (MAE) and the Root Mean Squared Error (RMSE), meaning its price predictions are substantially closer to reality and suffer from fewer extreme miscalculations.

Furthermore, the higher R^2 score confirms the Random Forest explains a greater percentage of the variance in the housing market.

Finally, extracting the feature importances allowed us to prove that core structural variables; specifically overall quality, above-ground living area (square footage), and neighborhood; are the dominant drivers of a property's final valuation in Ames.

```
%whos
```

```
Variable                Type                       Data/Info
-----------------------------------------------------------
LinearRegression        ABCMeta                    <class
'sklearn.linear_mo<...>._base.LinearRegression'>
RandomForestRegressor   ABCMeta                    <class
'sklearn.ensemble.<...>t.RandomForestRegressor'>
X                       DataFrame                        Order
PID  M<...>[2930 rows x 280 columns]
X_test                  DataFrame                        Order
PID  M<...>n[586 rows x 280 columns]
X_train                 DataFrame                        Order
PID  M<...>[2344 rows x 280 columns]
ax                      ndarray                    2: 2 elems, type
`object`, 16 bytes
comparison_df           DataFrame
<...>45.249191        0.916686
conn                    Connection                 <sqlite3.Connection
object at 0x7dba434f4c70>
cursor                  Cursor                     <sqlite3.Cursor
object at 0x7dba20b4ca40>
dataset_path            str
/content/drive/MyDrive/Am<...>g_Project/AmesHousing.csv
df                      DataFrame                        Order
PID  M<...>n[2930 rows x 84 columns]
df_ml                   DataFrame                        Order
PID  M<...>[2930 rows x 281 columns]
drive                   module                     <module
'google.colab.dri<...>s/google/colab/drive.py'>
feature_df              DataFrame
Featu<...>n\n[280 rows x 2 columns]
fig                     Figure                     Figure(1200x500)
importances             ndarray                    280: 280 elems, type
`float64`, 2240 bytes
lr_model                LinearRegression           LinearRegression()
mae_lr                  float                      16400.673170728554
mae_rf                  float                      15867.932067964919
mean_absolute_error     function                   <function
mean_absolute_error at 0x7dba222e7920>
mean_squared_error      function                   <function
mean_squared_error at 0x7dba222e7ce0>
missing_values          DataFrame                     total_records
```

```
missing<...>                    80
neighborhood_analysis   DataFrame                   Neighborhood
total_ho<...>   37           95756.49
np                      module                      <module 'numpy' from
'/us<...>kages/numpy/__init__.py'>
number_columns          Index                       Index(['Order',
'PID', 'M<...>],\n      dtype='object')
outliers                DataFrame                         Order
PID  M<...>\n\n[3 rows x 82 columns]
pd                      module                      <module 'pandas' from
'/u<...>ages/pandas/__init__.py'>
plot                    module                      <module
'matplotlib.pyplo<...>es/matplotlib/pyplot.py'>
plt                     module                      <module
'matplotlib.pyplo<...>es/matplotlib/pyplot.py'>
query                   str                         \nSELECT\n
COUNT(*) AS<...>ing\nFROM ames_housing;\n
r2_lr                   float                       0.8455768470687586
r2_rf                   float                       0.9166856361181789
r2_score                function                    <function r2_score at
0x7dba22274540>
residential_homes       DataFrame                         Order
PID  M<...>n[1891 rows x 82 columns]
rf_model                RandomForestRegressor
RandomForestRegressor(max<...>epth=10, random_state=42)
rmse_lr                 float64                     35186.57457423395
rmse_rf                 float64                     25845.249191207102
save_path               str
/content/drive/MyDrive/Am<...>_Project/ames_cleaned.csv
schema                  DataFrame                       cid
name  <...>\n\n[82 rows x 6 columns]
sns                     module                      <module 'seaborn'
from '/<...>ges/seaborn/__init__.py'>
sqlite3                 module                      <module 'sqlite3'
from '/<...>.13/sqlite3/__init__.py'>
text_columns            Index                       Index(['MS Zoning',
'Stre<...>],\n      dtype='object')
top5                    Series                      Neighborhood\nStoneBr
<...>SalePrice, dtype: float64
top_10_features         DataFrame                             Feature
Impor<...>  Garage Area    0.008282
top_correlations        DataFrame
SalePri<...>replaces         0.474558
train_test_split        function                    <function
train_test_split at 0x7dba2229e3e0>
y                       Series                      0       215000\n1
1<...>ength: 2930, dtype: int64
y_pred_lr               ndarray                     586: 586 elems, type
`float64`, 4688 bytes
```

```
y_pred_rf                ndarray                  586: 586 elems, type
`float64`, 4688 bytes
y_test                   Series                   1357    161000\n2367
1<...>Length: 586, dtype: int64
y_train                  Series                   381     152000\n834
1<...>ength: 2344, dtype: int64
```

```
df.to_csv("ames_cleaned.cvs", index=False)
```

```
import pandas as pd
predictions_df = pd.DataFrame({"SalePrice": y_test, "PredictedPrice":
y_pred_rf})

predictions_df["PredictionError"] = (predictions_df["SalePrice"] -
predictions_df["PredictedPrice"])

predictions_df.to_csv("ames_predictions.csv", index=False)
```

```
from google.colab import files

# The file was saved as .cvs in cell bRJVLcTnj0kt, so we download that
name
files.download("ames_cleaned.cvs")
print("ames_cleaned.cvs downloaded successfully")

files.download("ames_predictions.csv")
print("ames_predictions.csv downloaded successfully")
```

```
<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

ames_cleaned.cvs downloaded successfully

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

ames_predictions.csv downloaded successfully
```